



Diseño e Implementación de un Simulador de Segmentación Paginada para la Gestión de Memoria en Sistemas Operativos

Autores

Juan Diego Grajales Castillo

Steven Navarro Parrales

Alejandro Nuñez Barrera

Docente

Oswaldo Alberto Romero Villalobos

Universidad Distrital Francisco José de Caldas

Ingeniería de Sistemas

Sistemas Operativos

Bogotá, Colombia

04 de junio de 2026

Resumen

El presente informe documenta el diseño, la implementación y la evaluación de un simulador interactivo de **segmentación paginada** para un sistema con espacio de direccionamiento de 2^{32} bytes (4 GiB). El simulador fue desarrollado íntegramente en JavaScript y HTML5 para ejecutarse de forma nativa en el navegador web, sin dependencias externas. El sistema modela los cinco segmentos estándar de un proceso (Código, Datos, BSS, Heap y Stack), aplica la técnica de paginación sobre cada segmento y realiza la traducción completa de dirección lógica a dirección física mostrando la descomposición bit a bit de la dirección. El catálogo de procesos de prueba incluye cinco aplicaciones reales (NotePad, Word, Excel, AutoCad, Calculadora) con tamaños de segmento representativos. Los resultados demuestran que la combinación segmentación + paginación elimina la fragmentación externa y reduce la fragmentación interna al nivel de una sola página por segmento.

Índice

1. Introducción	5
2. Objetivos	5
2.1. Objetivo general	5
2.2. Objetivos específicos	6
3. Marco teórico	6
3.1. Espacio de direcciones y tamaño de la RAM	6
3.2. Segmentación	6
3.3. Paginación	7
3.4. Segmentación paginada	7
3.5. División de bits en el simulador	8
3.6. Fragmentación	8
4. Diseño del simulador	8
4.1. Arquitectura general	8
4.2. Estado global	9
4.3. Funciones auxiliares de bits	9
4.4. Asignador de marcos	10
4.5. Descarga de procesos y fragmentación	11
4.6. Traducción de dirección lógica a física	11
5. Implementación	12
5.1. Estructura de archivos	12
5.2. Catálogo de procesos de prueba	12
5.3. Persistencia del catálogo	13
5.4. Límites de visualización	13
6. Resultados	13
6.1. Verificación de la división de bits	13
6.2. Análisis de fragmentación interna	14
6.3. Ausencia de fragmentación externa	14
6.4. Comportamiento del simulador con los procesos de prueba	14
7. Manual de usuario	15
7.1. Requisitos de ejecución	15
7.2. Descripción de la interfaz	15
7.2.1. Zona Hero — estadísticas generales	15
7.2.2. Barra lateral — configuración y catálogo	16
7.2.3. Área principal — mapa de RAM y tabla de procesos	17

7.2.4. Panel de traducción de dirección lógica	18
7.2.5. Panel de explicación	19
7.3. Flujo de uso típico	19
8. Conclusiones	20

1. Introducción

La gestión de memoria es uno de los pilares fundamentales de cualquier sistema operativo moderno. Sin ella, múltiples procesos no podrían coexistir en la misma máquina de forma segura y eficiente. A lo largo de la evolución de los sistemas operativos se han desarrollado distintas estrategias para administrar la memoria física: asignación contigua, particiones fijas, particiones variables, segmentación, paginación y, como síntesis de las dos últimas, la **segmentación paginada** [1].

La **segmentación** otorga al programador una visión lógica de la memoria, dividiendo el espacio de direcciones de un proceso en regiones con significado semántico: el segmento de código contiene las instrucciones ejecutables; el segmento de datos, las variables globales inicializadas; el segmento BSS, las variables globales no inicializadas; el heap, la memoria dinámica; y el stack, las variables locales y los marcos de activación de funciones. Esta división refleja directamente la estructura de un programa compilado y facilita la protección de memoria entre regiones.

Sin embargo, la segmentación pura sufre de **fragmentación externa**: con el tiempo, el espacio libre de la memoria física queda fragmentado en huecos demasiado pequeños para alojar nuevos segmentos. La **paginación** resuelve este problema dividiendo tanto el espacio lógico como el espacio físico en bloques de tamaño fijo (páginas y marcos, respectivamente), eliminando por completo la fragmentación externa [2].

La **segmentación paginada** combina ambas técnicas: cada segmento lógico se divide a su vez en páginas, y cada página se mapea independientemente a un marco físico. Esto preserva la visión lógica de la segmentación para el programador y, al mismo tiempo, aprovecha las ventajas de la paginación para la asignación física de memoria [1].

El presente proyecto aborda la implementación de un simulador didáctico de segmentación paginada que permite visualizar de forma interactiva el proceso de traducción de direcciones lógicas a físicas en un sistema con espacio de direccionamiento de 2^{32} bits. El simulador opera completamente en el navegador web, utilizando JavaScript puro, lo que garantiza portabilidad y accesibilidad sin necesidad de instalar software adicional.

2. Objetivos

2.1. Objetivo general

Diseñar e implementar un simulador interactivo de segmentación paginada que modele la gestión de memoria de un sistema con 2^{32} bytes de RAM, permitiendo cargar procesos con múltiples segmentos, visualizar la asignación de marcos físicos y traducir direcciones lógicas a físicas bit a bit.

2.2. Objetivos específicos

1. Explicar cómo se divide el espacio de 2^{32} bits de direcciones lógicas en los campos de segmento, número de página y desplazamiento, y cómo varía esta división según el tamaño de página configurado.
2. Implementar el algoritmo de asignación de marcos físicos de forma eficiente, evitando la creación de estructuras de datos proporcionales al número total de marcos (2^{20} para páginas de 4 KiB).
3. Desarrollar un módulo de traducción de dirección lógica a física que muestre visualmente la descomposición bit a bit de la dirección y el proceso completo de consulta a la tabla de segmentos y a la tabla de páginas.
4. Construir una interfaz de usuario que represente gráficamente el mapa de marcos de RAM, las estadísticas de uso y las tablas de segmentos y páginas de cada proceso cargado.
5. Demostrar la ausencia de fragmentación externa y cuantificar la fragmentación interna máxima (tamaño de página – 1 bytes por segmento) mediante los procesos de prueba incluidos en el catálogo.

3. Marco teórico

3.1. Espacio de direcciones y tamaño de la RAM

Un sistema con buses de dirección de n bits puede direccionar hasta 2^n bytes de memoria. Para $n = 32$, esto equivale a:

$$2^{32} \text{ bytes} = 4,294,967,296 \text{ bytes} = 4 \text{ GiB}$$

Cada proceso en el simulador dispone de su propio espacio lógico de 2^{32} bytes, completamente independiente del espacio de otros procesos. La traducción de dirección lógica a física es responsabilidad de la Unidad de Manejo de Memoria (MMU), que en este simulador se emula por software.

3.2. Segmentación

En la segmentación, la dirección lógica se compone de un *número de segmento* s y un *desplazamiento* d :

$$\langle s, d \rangle$$

La **tabla de segmentos** asocia cada número de segmento con su *dirección base* en la memoria física y su *límite* (tamaño máximo). Si $d \geq \text{límite}[s]$, la MMU genera una excepción de protección de memoria (en sistemas reales, un *segmentation fault*).

La dirección física se calcula como:

$$\text{Dir. física} = \text{base}[s] + d$$

3.3. Paginación

En la paginación, tanto el espacio lógico como el físico se dividen en bloques de tamaño fijo P . Los bloques lógicos se llaman **páginas** y los bloques físicos se llaman **marcos**. La dirección lógica se descompone en:

$$\langle \underbrace{\quad}_{\text{número de página}}, \underbrace{\quad}_{\text{desplazamiento}} \rangle$$

donde $p = \lfloor \text{dir. lógica} / P \rfloor$ y $d = \text{dir. lógica} \bmod P$.

La **tabla de páginas** mapea cada número de página a su marco físico m :

$$\text{Dir. física} = m \times P + d$$

3.4. Segmentación paginada

La segmentación paginada combina ambas técnicas. La dirección lógica de n bits se divide en **tres campos**:

segcolor!20 Bits de segmento	Bits de página	Bits de offset
s bits	p bits	o bits

$$s + p + o = n = 32 \text{ bits}$$

El proceso de traducción sigue estos pasos:

1. Extraer el número de segmento S de los s bits más significativos.
2. Consultar la *tabla de segmentos* para obtener el número de página base del segmento (o un puntero a su tabla de páginas propia).
3. Extraer el número de página P de los siguientes p bits.
4. Consultar la *tabla de páginas del segmento* para obtener el marco físico M .
5. Extraer el desplazamiento d de los o bits menos significativos.
6. Calcular la dirección física: $\text{Dir. física} = M \times \text{tam. página} + d$.

3.5. División de bits en el simulador

El simulador fija $n = 32$ bits y distribuye los campos según el tamaño de página seleccionado. Los bits de segmento se fijan en $s = 3$ (para soportar hasta $2^3 = 8$ segmentos, usando 5 de ellos). Los bits de offset son $o = \log_2(\text{tam. página})$. Los bits de página son el resto:

$$p = 32 - s - o = 32 - 3 - o$$

Tabla 1: División del espacio de direcciones según el tamaño de página

Tam. página	Bits seg.	Bits pág.	Bits offset	Marcos totales	Págs. máx./seg.
4 KiB (2^{12})	3	17	12	1 048 576	131 072
8 KiB (2^{13})	3	16	13	524 288	65 536
16 KiB (2^{14})	3	15	14	262 144	32 768

Por ejemplo, con páginas de 4 KiB, la dirección lógica se representa como:

Segmento	Número de página	Offset
3 bits [31–29]	17 bits [28–12]	12 bits [11–0]

3.6. Fragmentación

Fragmentación externa: La segmentación paginada la elimina por completo, ya que cualquier marco libre puede asignarse a cualquier página.

Fragmentación interna: Puede ocurrir en la última página de cada segmento. Si un segmento de k bytes no es múltiplo del tamaño de página P , la última página desperdicia $P - (k \bmod P)$ bytes. El desperdicio máximo es $P - 1$ bytes por segmento (para páginas de 4 KiB: hasta 4 095 bytes, es decir, menos de 1 KiB por segmento).

4. Diseño del simulador

4.1. Arquitectura general

El simulador está compuesto por tres archivos:

index.html: Estructura semántica de la interfaz. Define las secciones Hero, Workspace (barra lateral + área principal), Traducción y Explicación.

styles.css: Hoja de estilos completa con esquema de color oscuro, variables CSS, diseño responsive y animaciones. No se utiliza ningún framework CSS externo.

`app.js`: Lógica completa del simulador: constantes, estado global, algoritmos de asignación de marcos, funciones de renderizado y manejadores de eventos.

La arquitectura sigue el patrón **MVC simplificado**: el estado global (`state`) actúa como modelo; las funciones `render*()` actúan como vistas que generan HTML dinámicamente; y los manejadores de eventos actúan como controladores que modifican el estado y llaman a `renderAll()`.

4.2. Estado global

Todo el estado de la simulación reside en un único objeto:

Listing 1: Estado global del simulador (`app.js`)

```
1 const state = {  
2   pageSize: 4096,    // tamaño de pagina en bytes (configurable)  
3   catalog: [],       // lista de procesos disponibles para cargar  
4   loaded: [],        // procesos actualmente en RAM  
5   // No se usa frameMap -- la propiedad de cada marco se  
6   // deriva de baseFrame + pageCount en cada segmento.  
7   nextFrame: 0,      // próximo marco libre  
8   nextLoadId: 1,     // ID unico de la siguiente instancia cargada  
9   colorIndex: 0,     // indice en el arreglo COLORS  
10 };
```

La decisión de no usar un arreglo `frameMap` es fundamental: para páginas de 4 KiB, la RAM de 4 GiB tiene $2^{20} = 1\,048\,576$ marcos. Un arreglo de ese tamaño con objetos JavaScript congelaría el navegador. En su lugar, la propiedad de cada marco se *deriva* consultando, para cada proceso cargado, si el marco está en el rango `[baseFrame, baseFrame + pageCount)`. Esta operación es $O(\text{procesos} \times \text{segmentos})$, no $O(\text{marcos})$.

4.3. Funciones auxiliares de bits

Listing 2: Cálculo de la distribución de bits

```
1 const LOGICAL_BITS = 32;  
2 const TOTAL_RAM    = 2 ** 32;    // 4 GiB  
3  
4 function totalFrames() { return Math.floor(TOTAL_RAM / state.pageSize); }  
5 function offBits()     { return Math.round(Math.log2(state.pageSize)); }  
6 function segBitsCount() { return 3; }  
7 function pgBits()      { return LOGICAL_BITS - segBitsCount() - offBits(); }  
8 function pageCnt(size) { return Math.ceil(size / state.pageSize); }
```

Por qué `Math.ceil` en `pageCnt`: La última página de un segmento puede estar parcialmente ocupada (fragmentación interna). Si un segmento tiene 5 000 bytes y el

tamaño de página es 4096 bytes, se necesitan $\lceil 5000/4096 \rceil = 2$ páginas; la segunda página ocupa solo $5000 - 4096 = 904$ bytes y desperdicia $4096 - 904 = 3192$ bytes.

4.4. Asignador de marcos

Listing 3: Asignación contigua de marcos dentro de un segmento

```
1 function allocFrames(count) {
2   const base = state.nextFrame;
3   state.nextFrame += count;
4   return base;
5 }
6
7 function loadProgram(entry) {
8   const tf = totalFrames();
9   const segs = [
10    { name: "Codigo", size: Math.max(1, +entry.txt) },
11    { name: "Datos", size: Math.max(1, +entry.data) },
12    { name: "BSS", size: Math.max(1, +entry.bss) },
13    { name: "Heap", size: Math.max(1, +entry.heap) },
14    { name: "Stack", size: Math.max(1, +entry.stack) },
15  ];
16  const needed = segs.reduce((s, g) => s + pageCnt(g.size), 0);
17  if (state.nextFrame + needed > tf) return null; // RAM insuficiente
18
19  const color = COLORS[state.colorIndex % COLORS.length];
20  state.colorIndex++;
21  const loadId = state.nextLoadId++;
22
23  const loadedSegs = segs.map((seg, i) => {
24    const pc = pageCnt(seg.size);
25    const baseFrame = allocFrames(pc);
26    return { index: i, name: seg.name, size: seg.size, baseFrame, pageCount: pc };
27  });
28
29  const prog = { loadId, name: entry.name, color, segments: loadedSegs,
30    totalPages: needed };
31  state.loaded.push(prog);
32  return prog;
33 }
```

El flujo de loadProgram es:

1. Calcular el total de páginas necesarias (*needed*).
2. Verificar que haya marcos libres suficientes.

3. Para cada segmento, llamar a `allocFrames(pc)` que devuelve el marco base y avanza `nextFrame`.
4. Guardar el proceso en `state.loaded`.

4.5. Descarga de procesos y fragmentación

Listing 4: Descarga de un proceso y recálculo de `nextFrame`

```

1 function unloadProgram(loadId) {
2   state.loaded = state.loaded.filter(p => p.loadId !== loadId);
3   // Recalcula nextFrame como el marco mas alto aun en uso.
4   // Recupera el espacio libre al final, pero deja huecos internos.
5   state.nextFrame = state.loaded.reduce((max, prog) =>
6     prog.segments.reduce(
7       (m, s) => Math.max(m, s.baseFrame + s.pageCount), max)
8     , 0);
9 }

```

Fragmentación tras descarga parcial: Si se cargan los procesos A, B, C en ese orden y luego se descarga B, los marcos de B quedan como *huecos* permanentes hasta que se ejecute `clearRAM()`. Este comportamiento modela con fidelidad el de un asignador de memoria lineal sin compactación. La compactación requeriría reubicar todos los marcos restantes y actualizar todas las tablas de páginas, operación análoga a la reubicación en tiempo de ejecución que realizaría un sistema real.

4.6. Traducción de dirección lógica a física

Listing 5: Algoritmo de traducción (`renderTranslationResult`)

```

1 const ob      = offBits();           // bits de offset
2 const pb      = pgBits();           // bits de pagina
3 const sb      = segBitsCount();      // bits de segmento (= 3)
4
5 const pageNum = Math.floor(offset / state.pageSize); // numero de pagina
6 const pageOff = offset % state.pageSize;           // desplazamiento
7 const frameNum = seg.baseFrame + pageNum;          // marco fisico
8 const physAddr = frameNum * state.pageSize + pageOff; // direccion fisica
9
10 // Reconstruccion de la direccion logica de 32 bits:
11 const logAddr  = (segIdx << (pb + ob)) | (pageNum << ob) | pageOff;

```

La Tabla 2 resume el proceso de traducción con un ejemplo concreto usando el proceso *NotePad*, segmento *Heap*, offset 1024.

Tabla 2: Ejemplo de traducción: NotePad · Heap · offset 1 024 (4 KiB/página)

Campo	Valor	Descripción
Índice de segmento	3 (Heap)	bits [31–29] = 011
Número de página	$\lfloor 1024/4096 \rfloor = 0$	bits [28–12] = 0000000000000000
Offset	$1024 \bmod 4096 = 1024$	bits [11–0] = 010000000000
Dirección lógica	0x60000400	$(3 \ll 29) \mid (0 \ll 12) \mid 1024$
Marco físico	<code>seg.baseFrame + 0</code>	depende del orden de carga
Dirección física	$\text{marco} \times 4096 + 1024$	calculada en tiempo de ejecución

5. Implementación

5.1. Estructura de archivos

```

proyecto/
  index.html      <- estructura HTML de la interfaz
  styles.css      <- hoja de estilos completa (856 líneas)
  app.js          <- logica completa del simulador (703 líneas)
  programas.json  <- catalogo inicial de procesos de prueba

```

5.2. Catálogo de procesos de prueba

El archivo `programas.json` define cinco procesos con tamaños de segmento representativos de aplicaciones reales:

Tabla 3: Catálogo de procesos incluidos en `programas.json`

Proceso	.txt (B)	.data (B)	.bss (B)	Heap (B)	Stack (B)	Total
NotePad	195 240	12 352	1 165	131 072	65 536	≈ 395 KiB
Word	775 390	32 680	4 100	262 144	65 536	≈ 1,1 MiB
Excel	995 420	24 245	7 557	524 288	65 536	≈ 1,6 MiB
AutoCad	1 150 000	123 470	1 123	1 048 576	131 072	≈ 2,4 MiB
Calculadora	123 420	1 246	1 756	131 072	32 768	≈ 285 KiB

Con todos los procesos cargados simultáneamente y tamaño de página de 4 KiB, el uso total de RAM es de aproximadamente 5,8 MiB de un total de 4 GiB, lo que representa el 0,00014 % de la RAM física. Esto ilustra por qué la paginación es eficiente: los marcos se asignan solo donde se necesitan.

5.3. Persistencia del catálogo

El catálogo se persiste en `localStorage` del navegador bajo la clave `secpag-v3-catalog`. Al arrancar, el simulador intenta leer el catálogo guardado; si no existe, intenta cargarlo desde `programas.json` vía `fetch`; si tampoco está disponible (por ejemplo, cuando se abre el archivo directamente con el protocolo `file://`), usa el catálogo por defecto embebido en el código.

Nota de ejecución: Para que `fetch()` funcione correctamente (y se cargue `programas.json`), el proyecto debe servirse desde un servidor HTTP local. Se recomienda usar la extensión *Live Server* de VS Code o ejecutar `python -m http.server 8080` en la carpeta del proyecto.

5.4. Límites de visualización

Para evitar bloquear el navegador al visualizar procesos con miles de páginas, el simulador define dos constantes de control:

Listing 6: Constantes de límite para la interfaz visual

```
1 const MAX_VISUAL_CELLS = 256; // max celdas de color por fila de segmento
2 const MAX_PT_ENTRIES   = 200; // max entradas en la tabla de paginas
   desplegable
```

Cuando un segmento tiene más de 256 páginas, cada celda visual representa $\lceil \text{pageCount}/256 \rceil$ páginas. El *tooltip* de cada celda indica el rango de páginas y marcos que representa. Esto permite visualizar segmentos de varios MiB (como el Heap de AutoCad, con 256 páginas de 4 KiB = 1 GiB teórico) sin degradación de rendimiento.

6. Resultados

6.1. Verificación de la división de bits

La Tabla 1 ya presentada confirma que para cada tamaño de página la suma de bits es exactamente 32. A continuación se verifica matemáticamente para el caso de 4 KiB:

$$s + p + o = 3 + 17 + 12 = 32 \checkmark$$

$$2^s = 8 \text{ segmentos posibles } (\geq 5 \text{ usados}) \checkmark$$

$$2^p = 2^{17} = 131\,072 \text{ páginas máximas por segmento } \checkmark$$

$$2^o = 2^{12} = 4\,096 \text{ bytes por página (= tamaño de página)} \checkmark$$

6.2. Análisis de fragmentación interna

Tabla 4: Fragmentación interna con páginas de 4 KiB (análisis por proceso)

Proceso	Segmento	Tamaño (B)	Páginas	Fragmentación int. máx. (B)
NotePad	Código	195 240	48	$4\,096 - (195\,240 \bmod 4\,096) = 3\,192$
	Datos	12 352	4	$4\,096 - (12\,352 \bmod 4\,096) = 0$
	BSS	1 165	1	$4\,096 - 1\,165 = 2\,931$
	Heap	131 072	32	0 (múltiplo exacto)
	Stack	65 536	16	0 (múltiplo exacto)
AutoCad	Código	1 150 000	281	$4\,096 - (1\,150\,000 \bmod 4\,096) = 944$
	Datos	123 470	31	$4\,096 - (123\,470 \bmod 4\,096) = 2\,178$
	BSS	1 123	1	$4\,096 - 1\,123 = 2\,973$
	Heap	1 048 576	256	0 (múltiplo exacto)
	Stack	131 072	32	0 (múltiplo exacto)

La fragmentación interna total en el peor caso es de $5 \times (\text{tam. página} - 1) = 5 \times 4\,095 = 20\,475$ bytes por proceso, es decir, menos de 20 KiB independientemente del tamaño del proceso. Para un proceso grande como AutoCad ($\approx 2,4$ MiB), esto representa menos del 0,83 % de desperdicio.

6.3. Ausencia de fragmentación externa

La fragmentación externa es imposible en segmentación paginada porque:

- Los marcos físicos son todos del mismo tamaño.
- Cualquier marco libre puede asignarse a cualquier página de cualquier segmento.
- No se requiere espacio contiguo en la memoria física para un proceso (a diferencia de la segmentación pura).

6.4. Comportamiento del simulador con los procesos de prueba

Al cargar los cinco procesos del catálogo en orden (NotePad, Word, Excel, AutoCad, Calculadora) con tamaño de página de 4 KiB:

Tabla 5: Marcos asignados por proceso con páginas de 4 KiB

Proceso	Páginas totales	Memoria física usada	Marco base
NotePad	101	404 KiB	0
Word	271	1 084 KiB	101
Excel	398	1 592 KiB	372
AutoCad	601	2 404 KiB	770
Calculadora	72	288 KiB	1 371
Total	1 443	5 772 KiB ($\approx$ 5,6 MiB)	—

El porcentaje de uso de RAM es $1443/1\,048\,576 \approx 0,0014\%$, demostrando que la paginación permite una asignación granular y eficiente sin desperdiciar grandes bloques de memoria física.

7. Manual de usuario

7.1. Requisitos de ejecución

- Navegador web moderno (Chrome 90+, Firefox 88+, Edge 90+, Safari 14+).
- Servidor HTTP local para que funcione la carga de `programas.json`. Recomendado: extensión *Live Server* de VS Code, o el comando `python -m http.server 8080`.
- No se requieren dependencias externas, frameworks, ni instalación de software.

7.2. Descripción de la interfaz

La interfaz se divide en cuatro zonas principales que se describen a continuación.

7.2.1. Zona Hero — estadísticas generales

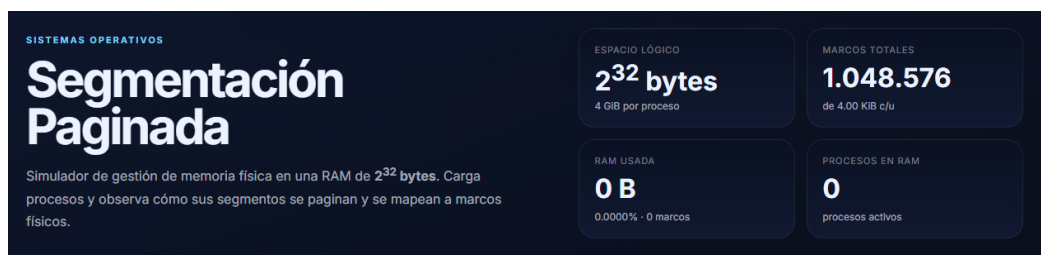


Figura 1: Zona Hero: estadísticas generales del sistema.

La zona Hero muestra cuatro tarjetas de estadísticas en tiempo real (Figura 1):

1. **Espacio lógico:** 2^{32} bytes (4 GiB por proceso).
2. **Marcos totales:** número de marcos en la RAM física según el tamaño de página activo.
3. **RAM usada:** bytes y porcentaje de marcos ocupados.
4. **Procesos en RAM:** número de procesos actualmente cargados.

7.2.2. Barra lateral — configuración y catálogo



Figura 2: Barra lateral: selector de tamaño de página, desglose de bits, catálogo de procesos y formulario para agregar proceso nuevo.

La barra lateral contiene tres paneles (Figura 2):

Panel Configuración. Permite seleccionar el tamaño de página (4 KiB, 8 KiB o 16 KiB). Debajo del selector se muestran los chips de bits en colores: amarillo para el campo de segmento, azul para el campo de página y verde para el campo de offset. El cambio de tamaño de página solo está permitido si no hay procesos cargados.

Panel Catálogo de procesos. Lista los procesos disponibles. Cada tarjeta muestra el nombre, el tamaño total y el desglose por segmento. Los botones **Cargar en RAM** y **Eliminar** permiten gestionar el catálogo. También incluye los botones **Limpiar RAM** y **Exportar JSON**.

Panel Nuevo proceso. Formulario para definir un proceso personalizado con nombre y tamaño de cada uno de los cinco segmentos en bytes.

7.2.3. Área principal — mapa de RAM y tabla de procesos

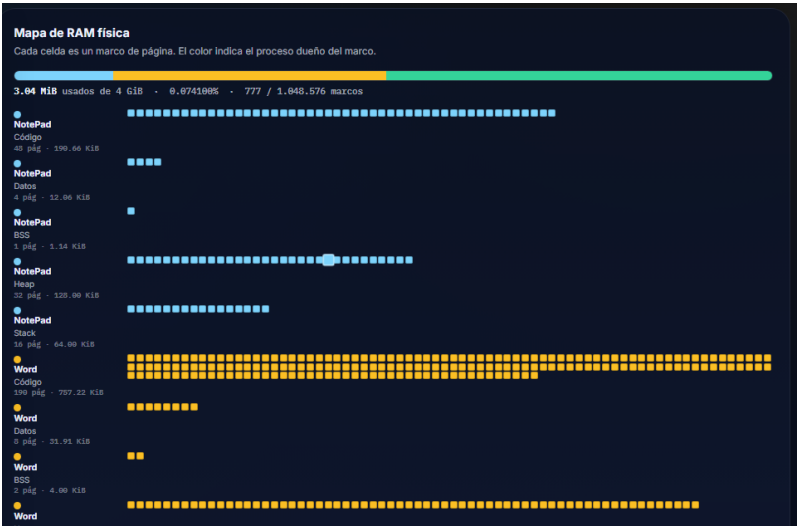


Figura 3: Mapa de RAM física: barra de uso proporcional, mapa de marcos por segmento y leyenda de colores. Se muestran tres procesos cargados.

El mapa de RAM (Figura 3) presenta:

- **Barra de uso:** muestra proporcionalmente cuánto RAM está ocupada, con un segmento de color por cada segmento de cada proceso.
- **Cuadrícula de marcos:** cada fila corresponde a un segmento de un proceso. Los pequeños cuadrados coloreados representan marcos físicos. Al pasar el cursor sobre una celda, el tooltip indica el rango de páginas y la dirección física de inicio.
- **Leyenda:** asocia cada color con el proceso propietario y muestra el total de marcos y bytes asignados.

NotePad 191 marcos - 804.00 KiB Descargar

SEG#	NOMBRE	TAMAÑO	PÁGINAS	BASE FÍSICA	LÍMITE FÍSICO	
0	Código	190.66 KiB	48	0x00000000	0x0002FFFF	▼ páginas
1	Datos	12.06 KiB	4	0x00030000	0x00033FFF	▼ páginas
2	BSS	1.14 KiB	1	0x00034000	0x00034FFF	▼ páginas
3	Heap	128.00 KiB	32	0x00035000	0x00054FFF	▼ páginas
4	Stack	64.00 KiB	16	0x00055000	0x00064FFF	▼ páginas

Figura 4: Tabla de segmentos de un proceso cargado (NotePad). Se muestra la tabla con los 5 segmentos y, desplegada, la tabla de páginas del segmento Heap.

La tabla de procesos (Figura 4) muestra, para cada proceso:

- El número de segmento, nombre, tamaño, cantidad de páginas y rango de direcciones físicas (base y límite).
- El botón **páginas** despliega la tabla de páginas del segmento, mostrando la correspondencia $P_i \rightarrow M_j$ para cada página.
- El botón **Descargar** elimina el proceso de la RAM.

7.2.4. Panel de traducción de dirección lógica

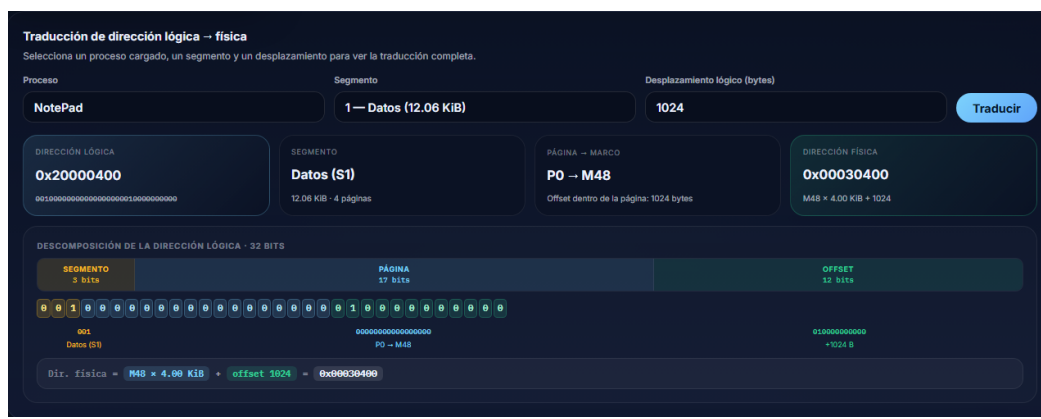


Figura 5: Panel de traducción: resultado completo para el proceso Word, segmento Heap, offset 8192. Se muestran las cuatro tarjetas (dirección lógica, segmento, página→marco, dirección física) y el diagrama de bits coloreado.

El panel de traducción (Figura 5) es el componente más importante del simulador. Para usarlo:

1. Seleccionar un proceso cargado en el desplegable **Proceso**.
2. Seleccionar el segmento deseado en **Segmento**.
3. Ingresar el desplazamiento lógico en bytes en **Desplazamiento lógico**. El campo valida automáticamente que el valor esté dentro del rango del segmento seleccionado.
4. Hacer clic en **Traducir**.

El resultado muestra:

- La **dirección lógica** en hexadecimal y en binario (32 bits).
- El **segmento** identificado y sus propiedades.

- La correspondencia **página** $\rightarrow$ **marco**.
- La **dirección física** resultante en hexadecimal.
- Un **diagrama de bits** que colorea cada campo (amarillo: segmento, azul: página, verde: offset) y muestra el valor binario de cada campo junto con su interpretación.
- La **fórmula** de cálculo: $\text{Dir. física} = M_k \times \text{tam. página} + \text{offset}$.

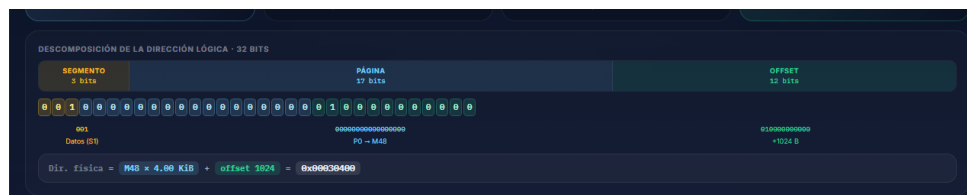


Figura 6: Diagrama de descomposición de bits de la dirección lógica. Los bits amarillos corresponden al campo de segmento, los azules al número de página y los verdes al offset.

7.2.5. Panel de explicación

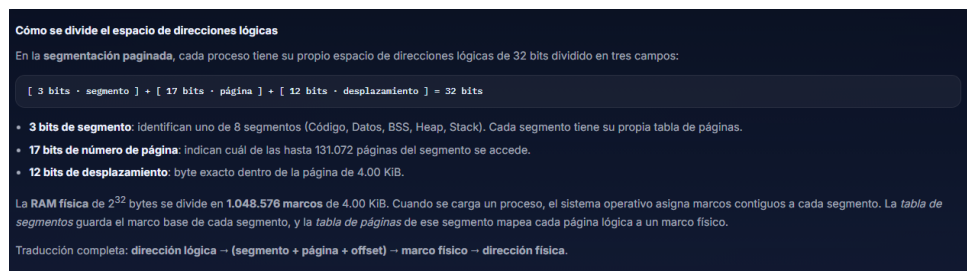


Figura 7: Panel de explicación: descripción dinámica de la división del espacio de direcciones según el tamaño de página activo.

El panel inferior (Figura 7) actualiza dinámicamente la descripción de la división del espacio de direcciones según el tamaño de página activo, mostrando la fórmula bit a bit y la capacidad máxima de cada campo.

7.3. Flujo de uso típico

1. Abrir el proyecto con un servidor HTTP local.
2. (Opcional) Cambiar el tamaño de página en el panel *Configuración*.
3. Hacer clic en **Cargar en RAM** para uno o más procesos del catálogo.
4. Observar cómo se actualiza el mapa de RAM y las estadísticas.

5. En el panel *Procesos en RAM*, hacer clic en **páginas** para ver la tabla de páginas de un segmento.
6. En el panel *Traducción*, seleccionar proceso, segmento y offset, y hacer clic en **Traducir** para ver el proceso completo paso a paso.
7. Usar **Descargar** para retirar un proceso individual o **Limpiar RAM** para reiniciar la simulación.

8. Conclusiones

1. **La segmentación paginada es una síntesis eficaz de dos técnicas complementarias.** La segmentación proporciona una visión lógica y semántica del espacio de un proceso, dividiendo el programa en regiones con propósito bien definido (código, datos, BSS, heap, stack). La paginación elimina la fragmentación externa al asignar marcos de tamaño fijo de forma independiente. La combinación de ambas aprovecha las ventajas de cada una sin heredar sus principales desventajas.
2. **La implementación eficiente de la tabla de marcos es crítica.** Para un sistema de 4 GiB con páginas de 4 KiB existen 2^{20} marcos. Mantener un arreglo con un objeto por marco haría inviable la simulación en el navegador. La decisión de derivar la propiedad de cada marco a partir de los campos `baseFrame` y `pageCount` almacenados en cada segmento reduce el consumo de memoria a $O(N_{\text{segmentos}})$, haciendo que el simulador maneje correctamente incluso procesos de varios GiB.
3. **La fragmentación interna es el único tipo de desperdicio en segmentación paginada, y está acotada.** El análisis cuantitativo demuestra que el desperdicio máximo es de $P - 1$ bytes por segmento, independientemente del tamaño del proceso. Para 5 segmentos y páginas de 4 KiB, el peor caso es $5 \times 4095 = 20475$ bytes por proceso, lo cual es despreciable frente al tamaño típico de los procesos.
4. **La visualización bit a bit fortalece la comprensión de la traducción de direcciones.** El diagrama de bits coloreado (amarillo: segmento, azul: página, verde: offset) permite ver de forma inmediata cómo los 32 bits de una dirección lógica se descomponen y cómo cada campo es utilizado en el proceso de traducción. Esta representación visual es más eficaz pedagógicamente que una descripción puramente textual.
5. **El simulador modela con fidelidad el comportamiento de un asignador de memoria lineal sin compactación.** La fragmentación que queda

tras descargar un proceso intermedio no se recupera hasta limpiar toda la RAM, lo que refleja el comportamiento de un sistema operativo real que no realiza compactación en línea por su alto costo (requeriría copiar marcos y actualizar TLBs en todos los procesadores activos).

6. **JavaScript puro es una plataforma viable para la simulación de sistemas operativos a nivel didáctico.** La capacidad del lenguaje para manejar enteros de 32 bits mediante operadores bit a bit, combinada con la potencia del DOM para la generación dinámica de la interfaz, permite construir una herramienta educativa completa, portable y sin dependencias externas.

Referencias

- [1] Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley.
- [2] Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4th ed.). Pearson.
- [3] Stallings, W. (2018). *Operating Systems: Internals and Design Principles* (9th ed.). Pearson.
- [4] Patterson, D. A., & Hennessy, J. L. (2017). *Computer Organization and Design: The Hardware/Software Interface* (5th ed.). Morgan Kaufmann.
- [5] Love, R. (2010). *Linux Kernel Development* (3rd ed.). Addison-Wesley.
- [6] Bovet, D. P., & Cesati, M. (2005). *Understanding the Linux Kernel* (3rd ed.). O'Reilly Media.